ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO



Context Management and On-Premises Monitoring

INSISN – Integração de Sistemas e Serviços na Nuvem

António Barros

Departmento de Engenharia Informática

FIWARE Orion Context Broker and IoT Agents

Challenges in Edge-Cloud Integration

- **Industrial Protocol Fragmentation:**
  - Edge devices communicate via heterogeneous protocols (MQTT, CoAP, Modbus, OPC UA).
  - Lack of unified interfaces forces custom point-to-point integrations.
- **State vs. Historical Data Disconnect:**
  - Operational systems require immediate awareness of current operational context.
  - Analytical tools require time-series persistence for trend evaluation and forecasting.
- **System Decoupling & Scalability:**
  - Direct coupling between sensors and analytical applications creates brittle architectures.
  - Need for an event-driven intermediate layer that isolates producers from consumers.

Generic Pattern: Decoupled Edge-Cloud Data Pipeline

- **Layer 1: Edge Abstraction and Adapter Pattern**
  - Normalises proprietary/fieldbus protocols into a common data exchange format.
  - Shields core management applications from hardware-level variations.
- **Layer 2: Central State Broker**
  - Serves as an in-memory single source of truth for current asset states.
  - Disseminates real-time state changes to registered downstream services.
- **Layer 3: Specialized Downstream Handlers**
  - **Time-Series Sink:** Asynchronously logs state history for analytics.
  - **Actuation & Control:** Consumes real-time alerts for local decision-making.

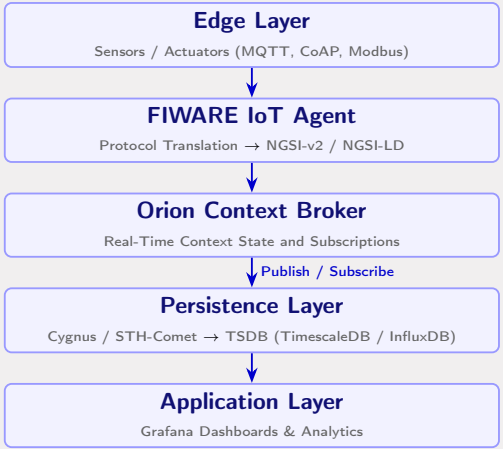
Concrete Implementation: The FIWARE Ecosystem

- **Standardizing the Pattern with Open Specifications:**
  - **Adapter Pattern** → FIWARE IoT Agents (Protocol Translation).
  - **State Broker** → Orion Context Broker (ETSI NGSI-v2 / NGSI-LD).
  - **Time-Series Sink** → Cygnus / STH-Comet connectors + TSDBs.
- **Key Architectural Benefits:**
  - **Interoperability:** Vendor-neutral abstraction based on European open standards.
  - **Modularity:** Replace or expand database sinks without modifying edge devices.
  - **Event-Driven Efficiency:** Native push notifications eliminate unnecessary polling.

Edge-Cloud Reference Architecture

- **Edge Layer:** Sensors, actuators, and protocol gateways handling local data collection.
- **IoT Agent Layer:** Protocol translation bridging industrial protocols to NGSI specifications.
- **Context Management Layer:** Orion Context Broker managing real-time entity state.
- **Historical Storage Layer:** Time-Series Databases (TSDB) storing persistent metrics.
- **Application Layer:** Grafana and analytical tools for visualization and decision-making.

Edge-Cloud Reference Architecture: Data Flow



ISEP

Separation of Architectural Responsibilities

- **Local Processing (Edge):**
  - Low-latency actuation and protocol adaptation.
  - Filtering and preprocessing high-frequency raw sensor streams.
- **Real-Time State Management (Context Broker):**
  - Single source of truth for current state (*What is happening now?*).
  - In-memory operations optimized for rapid state updates and queries.
- **Historical Storage (Persistence):**
  - Long-term trend analysis, auditing, and batch analytics.
  - Optimized for time-series range queries rather than key-value state lookups.

Fundamentals of Context Management

- **Context Information Defined:**
  - Represents the state of physical or logical entities at a given instant.
  - Goes beyond raw sensor values to include metadata (e.g., location, units, quality).
- **Communication Paradigms:**
  - **Traditional Polling:** Inefficient, high overhead, delays state detection.
  - **Publish-Subscribe:** Event-driven architecture; updates consumers automatically upon change.
- **Central Role of Orion Context Broker:**
  - Central hub of the FIWARE ecosystem.
  - Decouples data producers (IoT devices) from data consumers (analytics, UI).

NGSI Data Models: Entity Structure

- **Entity:** Represents a real-world object (e.g., `IndustrialSensor`, `Machine`).
- **Identifier (`id`):** Unique URI string within the context system.
- **Type (`type`):** Class/category defining the entity schema.
- **Attributes (`attributes`):** Key-value pairs representing state variables.
- **Metadata (`metadata`):** Properties of attributes (e.g., timestamps, measurement units).

NGSI Data Models: NGSI-v2 Payload Example

```
{
  "id": "urn:ngsi-ld:IndustrialSensor:001",
  "type": "IndustrialSensor",
  "temperature": {
    "value": 24.5,
    "type": "Float",
    "metadata": {
      "unitCode": {
        "type": "Text",
        "value": "CEL"
      }
    }
  },
  "status": {
    "value": "OK",
    "type": "Text"
  }
}
```

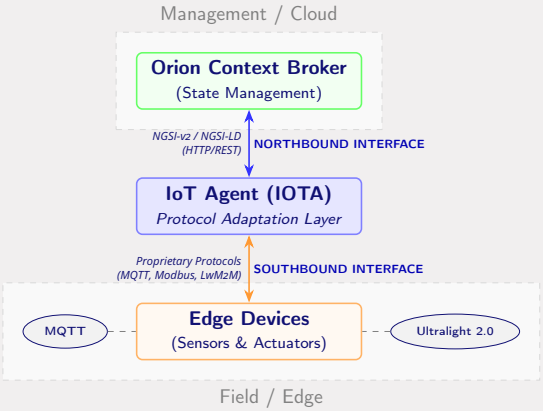
NGSI Specifications: NGSI-v2 vs. NGSI-LD

- **NGSI-v2:**
  - Simple JSON-based format optimized for rapid real-time operations.
  - Ideal for constrained environments and straightforward IoT deployments.
- **NGSI-LD:**
  - Based on JSON-LD (Linked Data) standard.
  - Incorporates **formal ontologies** and **semantic relationships** between entities.
  - Enables cross-domain interoperability and graph-based context traversal.

Bridging Edge and Broker: FIWARE IoT Agents

- **The Edge Heterogeneity Problem:**
  - Industrial environments use diverse, non-NGSI protocols (MQTT, CoAP, Modbus).
  - Bandwidth constraints and light payloads (e.g., Ultralight 2.0).
- **IoT Agent (IOTA) Architecture:**
  - **Southbound Interface:** Listens to device protocols (e.g., MQTT topics, HTTP endpoints).
  - **Northbound Interface:** Translates incoming raw data into NGSI operations for Orion.

IoT Agent Architecture: Northbound vs. Southbound



ISEP

IoT Agent: Device Provisioning and Data Translation

- **Device Provisioning:** Associates a physical device with its corresponding NGSI entity and defines how the device is accessed through its IoT protocol.
- **Data Mapping:** Translates protocol-specific measurements into NGSI attributes, e.g. `t=25` → `temperature=25`.
- **Bidirectional Communication:**
  - **Telemetry (upstream):** Device → IoT Agent → Context Broker
  - **Commands (downstream):** Context Broker → IoT Agent → Device
- **Abstraction:** Hides device-specific protocols from applications and the Context Broker by exposing device data through the NGSI interface.

Context Operations in Orion Context Broker 1/2

- **Context Representation:**
  - Orion stores the current state of **NGSI entities**, identified by an `id` and `type`.
  - Each entity contains a set of **attributes** representing its current context.
- **Create / Update Context:**
  - `POST /v2/entities` — Creates a new entity and its initial attributes.
  - `PATCH /v2/entities/{id}/attrs` — Updates selected attributes without replacing the complete entity.
  - Updates can originate from IoT Agents, applications, or other context-aware components.

Context Operations in Orion Context Broker 2/2

- **Query Current State:**
  - `GET /v2/entities/{id}` — Retrieves the current context of a specific entity.
  - Queries can select entities by **type**, attributes, or other context-related criteria.
- **Role in the Architecture:**
  - Orion provides a **central access point to current context information**.
  - Applications do not need to communicate directly with individual IoT devices to obtain their state.

Context Operations in Orion Context Broker

Create Context Example

```
$ curl localhost:1026/v2/entities -X POST -H 'Content-Type: application/json' -i -s -S \
-d '{
  "id": "urn:ngsi-ld:IndustrialSensor:001",
  "type": "IndustrialSensor",
  "temperature": {
    "value": 24.5, "type": "Float",
    "metadata": { "unitCode": { "type": "Text", "value": "CEL" } }
  },
  "status": {"value": "OK", "type": "Text"}
}'
```

-i : Include HTTP response headers in the output
-s : Silent mode (no progress or error messages)
-S : Show error messages if they occur

ISEP

Context Operations in Orion Context Broker

Update Context Example

```
$ curl localhost:1026/v2/entities/urn:ngsi-ld:IndustrialSensor:001/attrs -X PATCH \
-H 'Content-Type: application/json' -i -s -S \
-d '{
  "temperature": {
    "value": 27,
    "type": "Float",
    "metadata": {
      "unitCode": {
        "type": "Text",
        "value": "CEL"
      }
    }
  }
}'
```


ISEP

Context Operations in Orion Context Broker

Query Context Example

```
$ curl localhost:1026/v2/entities/urn:ngsi-ld:IndustrialSensor:001 \
-H 'Accept: application/json' -i -s -S \
```

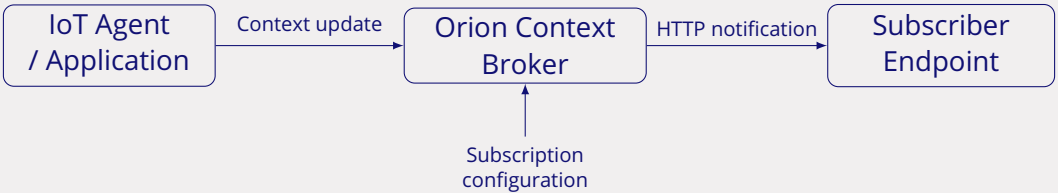
Find more information about the NGSIv2 API in the official documentation:

https://fiware-orion.readthedocs.io/en/latest/user/walkthrough_apiv2/index.html

Orion Subscriptions: Publish–Subscribe

- **Purpose:** Applications can subscribe to changes in context information without continuously polling the Context Broker.
- **Subscription:** Defines *what* entities and attributes are relevant and *where* notifications should be delivered.
- **Notification:** When a matching context update occurs, Orion automatically sends an HTTP notification to the subscribed endpoint.

Orion Subscriptions: Publish–Subscribe



Key idea

Orion acts as the event broker: subscribers do not need to poll for changes; Orion pushes notifications when the subscription conditions are satisfied.

Creating an NGSIv2 Subscription

- A subscription is created through the Orion REST API.
- The client sends an HTTP `POST` request to endpoint `/v2/subscriptions` with a JSON body describing the subscription.
- The request body specifies:
  - **Subject:** which entities and attributes to monitor;
  - **Condition:** which changes trigger a notification;
  - **Notification:** where and what data is sent.

Subscription lifecycle

`POST /v2/subscriptions` → Orion stores subscription → matching update → HTTP notification

NGSIv2 Subscriptions: Defining What to Notify

```
{
  "description": "Notify TSDB Persister",
  "subject": {
    "entities": [{
      "idPattern": ".*",
      "type": "IndustrialSensor"
    }],
    "condition": {
      "attrs": ["temperature"]
    }
  },
  "notification": {
    "http": {
      "url": "http://sth-comet:8666/notify"
    },
    "attrs": ["temperature"]
  }
}
```

1. Subject
Selects the context entities to monitor.

2. Condition
Specifies which attribute changes trigger a notification.

3. Notification
Defines the HTTP endpoint and the attributes included in the notification.

Example

Any `IndustrialSensor` whose `temperature` changes triggers an HTTP notification to the configured endpoint.

ISEP

On-Premises Data Persistence and Visualisation

ISEP

From Context to Historical Data

- **Orion stores current context:**
  - Maintains the latest state of NGSI entities and attributes.
  - Optimised for context updates, queries and subscriptions.
  - Not intended to provide long-term time-series analysis.
- **Historical persistence:**
  - Context changes are delivered through Orion subscriptions.
  - A persistence component consumes these notifications and writes them to a historical data store.

From Context to Historical Data

- **FIWARE persistence options:**
  - **STH-Comet:** specialised for storing and querying historical context information.
  - **Cygnus:** generic context-data sink supporting multiple persistence technologies.
  - **Custom connectors:** application-specific pipelines for specialised storage or processing requirements.

Persistence Connector: a component that subscribes to Orion notifications and writes the received context changes to a historical data store.



On-Premises Time-Series Databases

- **Time-series data model:**
  - Measurements are associated with a timestamp and one or more dimensions/tags.
  - Data is naturally queried over time windows.
  - Retention and downsampling policies control storage growth.

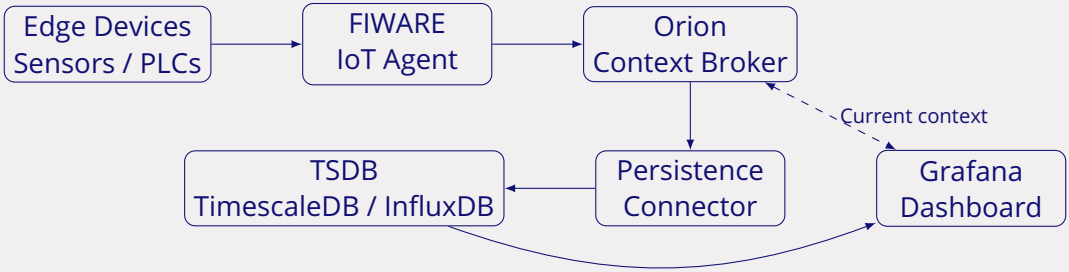
On-Premises Time-Series Databases

Characteristic	TimescaleDB	InfluxDB
Storage model	PostgreSQL / hypertables	Time-series engine
Query language	SQL	InfluxQL / Flux*
Relational joins	Native	Limited / model-dependent
Time-series optimisation	Hypertables, chunks	Time-series storage engine
Retention	Policies / jobs	Retention policies
Ecosystem	PostgreSQL ecosystem	Time-series ecosystem

* Depending on the InfluxDB version and deployment model.

High-frequency Edge data: Prefer batched writes, appropriate indexing, controlled cardinality and time-based partitioning. Avoid creating excessive indexes or issuing one transaction per measurement.

On-Premises Monitoring Architecture



ISEP

On-Premises Monitoring Architecture

- **Local operation:** Monitoring and historical analysis remain available within the on-premises network.
- **Low latency:** Local storage and processing minimise the network path for operational queries and dashboards.
- **Resilience:** Temporary Cloud connectivity failures do not necessarily interrupt local monitoring.

Visualisation and Dashboarding with Grafana

- **Grafana as a unified presentation layer:**
  - Connects to multiple data sources through plugins.
  - Provides a common interface for operational monitoring.
- **Data sources:**
  - **TimescaleDB:** queried using SQL.
  - **InfluxDB:** queried using the corresponding time-series query language/API.
- **Time-aware queries:**
  - Real-time view: latest measurements within a short time window.
  - Historical view: aggregation over longer periods.
  - Typical functions: `AVG`, `MAX`, `MIN`, rates and time-window aggregations.

Building Operational Dashboards

- **Panel selection depends on the operational question:**
  - **Time-series:** evolution of temperature, pressure, energy consumption, etc.
  - **Stat:** current value or key performance indicator.
  - **Gauge:** current value against a defined range.
  - **Heatmap:** distribution and temporal patterns.
- **Alerting:**
  - Define rules based on thresholds or derived metrics.
  - Example: trigger an alert when `temperature` $> T_{\text{critical}}$.
  - Alerts should distinguish warning conditions from critical operational states.

Building Operational Dashboards

- **Operational design principles:**
  - Prioritise information relevant to the operator.
  - Use consistent units, labels and time ranges.
  - Make abnormal conditions immediately distinguishable.
  - Avoid excessive panels and unnecessary visual complexity.